Universidad Tecnológica del Perú

BASE DE DATOS

Mg.Ing. Yuri A. Pomachagua Basualdo

Daisuke Kishimoto

2025

Desaprende lo que te limita

Universidad Tecnológica del Perú

Commit y Rollback

Desaprende lo que te limita

Ing Yuri A. Pomachagua Basualdo

Agenda

- Manejo de transacciones:
  - Commit y
  - Rollback

VectorStock® VectorStock.com/30487804

Logro

Al finalizar la sesión el estudiante conoce las transacciones Commit y CollBack, para aplicar en las consultas de la base de datos.

Universidad Tecnológica del Perú

https://daa1df3k0xsds.cloudfront.net/wp-content/uploads/2023/12/transacciones-electronicas-min-scaled.jpg

Utilidad

El manejo de transacciones es importante en SQL porque permite garantizar la integridad de los datos. Es decir garantiza que los datos permanezcan inalterados, completos y válidos, protegiéndolos de modificaciones no autorizadas, corrupción o pérdida

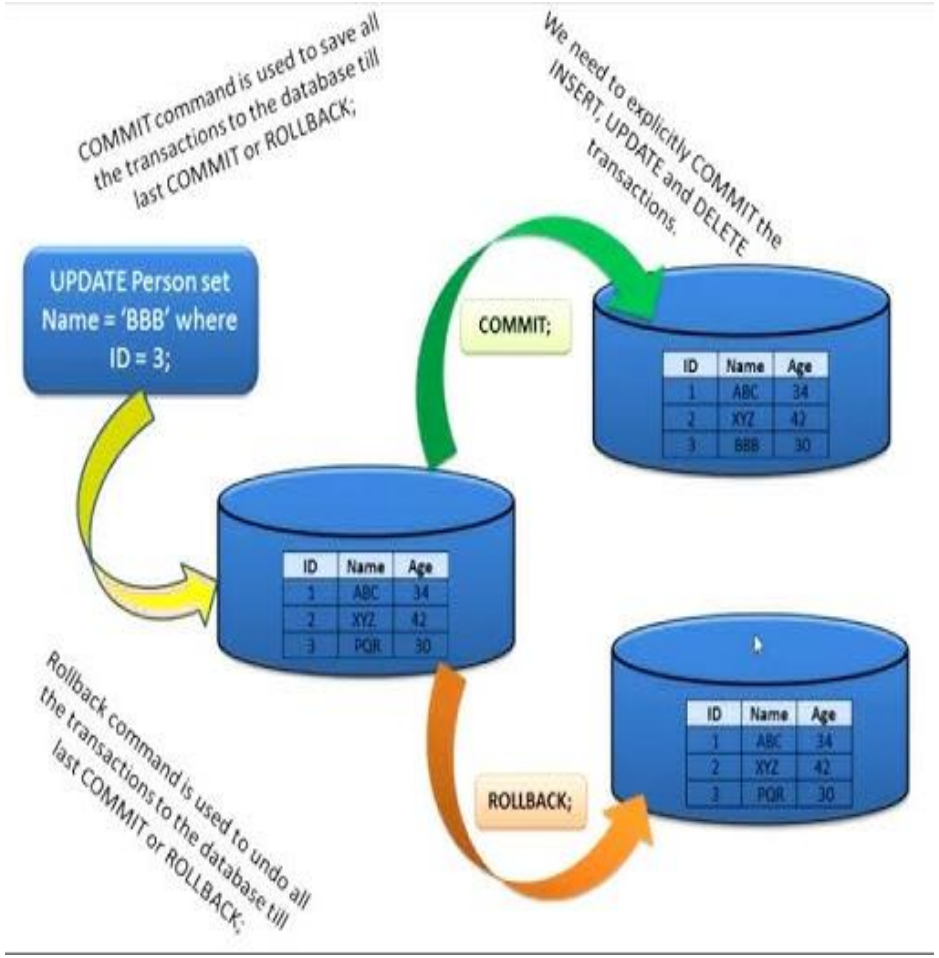
Universidad Tecnológica del Perú

Ing Yuri A. Pomachagua Basualdo

Manejo de Transacciones

Universidad Tecnológica del Perú

DEFINICIÓN



El **manejo de transacciones** es fundamental en SQL para garantizar la integridad de los datos. Las transacciones permiten agrupar varias operaciones SQL en una unidad lógica de trabajo que debe ejecutarse completamente o no ejecutarse en absoluto.

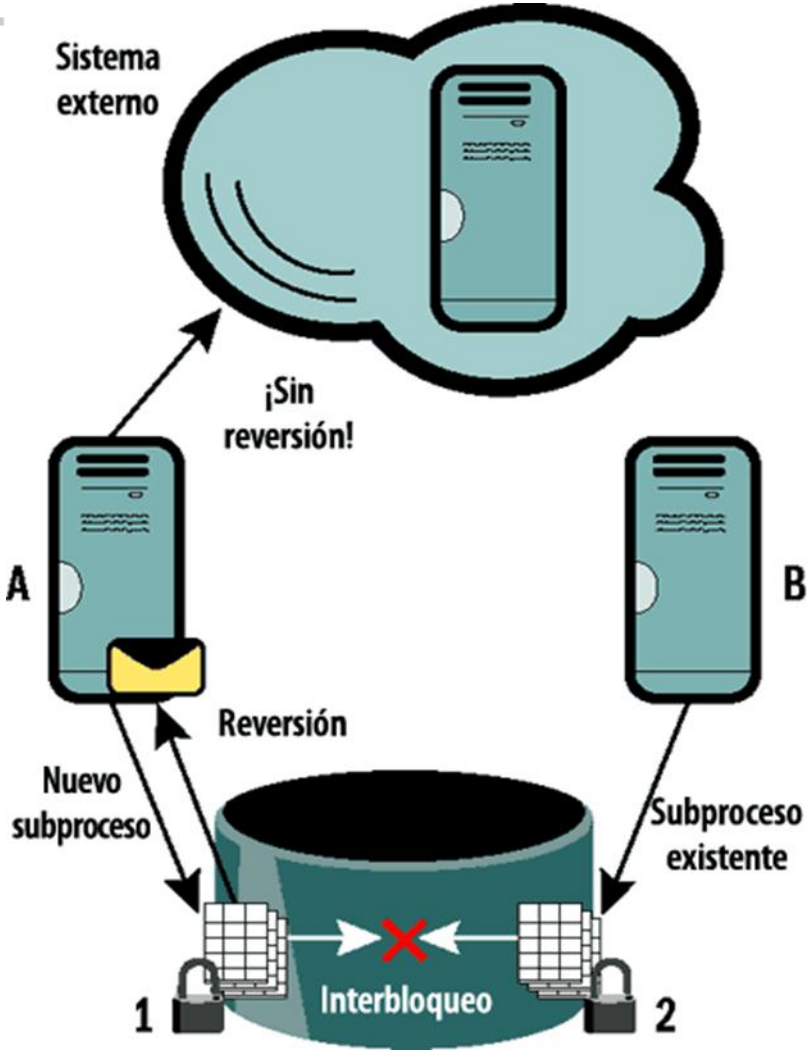
En SQL Server, las transacciones se controlan principalmente con las instrucciones BEGIN TRANSACTION, COMMIT y ROLLBACK.

Manejo de Transacciones

¿Qué es una Transacción?

Una transacción es una secuencia de operaciones que se tratan como una sola unidad. Se utiliza para garantizar que la base de datos permanezca en un estado coherente incluso si ocurre un error.

Manejo de Transacciones



Las propiedades ACID

Atomicidad: Todas las operaciones dentro de una transacción deben completarse. Si una falla, todo se revierte.

Consistencia: La base de datos pasa de un estado válido a otro válido.

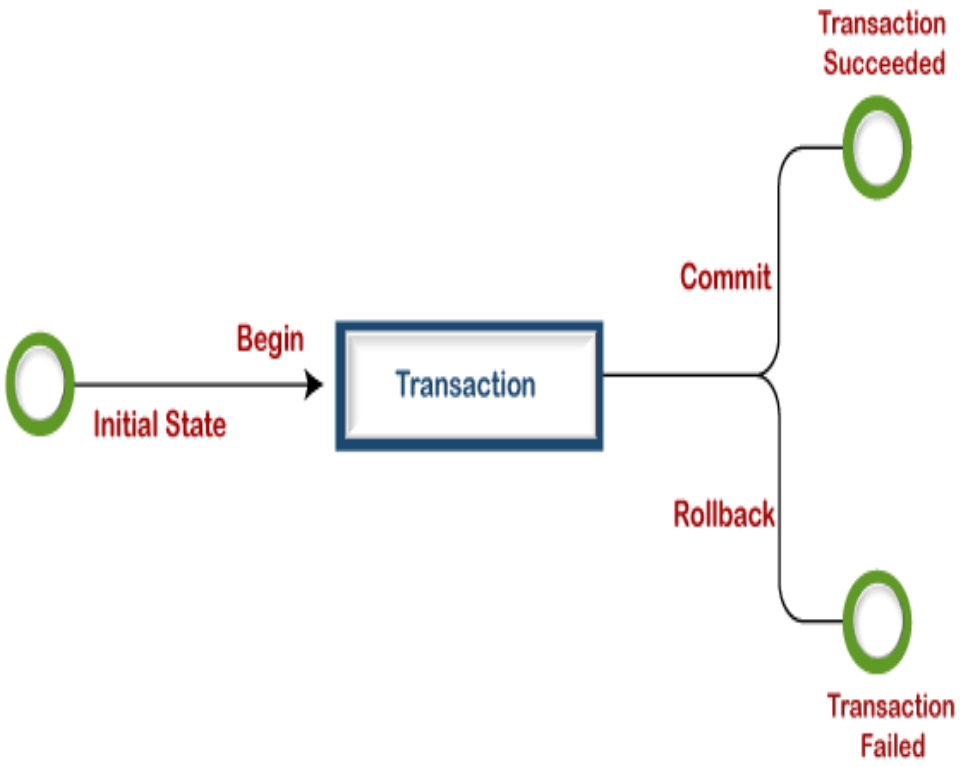
Aislamiento: Las transacciones concurrentes no interfieren entre sí.

Durabilidad: Una vez confirmada, la transacción se guarda permanentemente

Manejo de Transacciones

Universidad Tecnológica del Perú

Comandos Principales



- **BEGIN TRANSACTION**

  Inicia una nueva transacción.

- **COMMIT**

  Confirma los cambios realizados durante la transacción. Los datos se guardan de forma permanente.

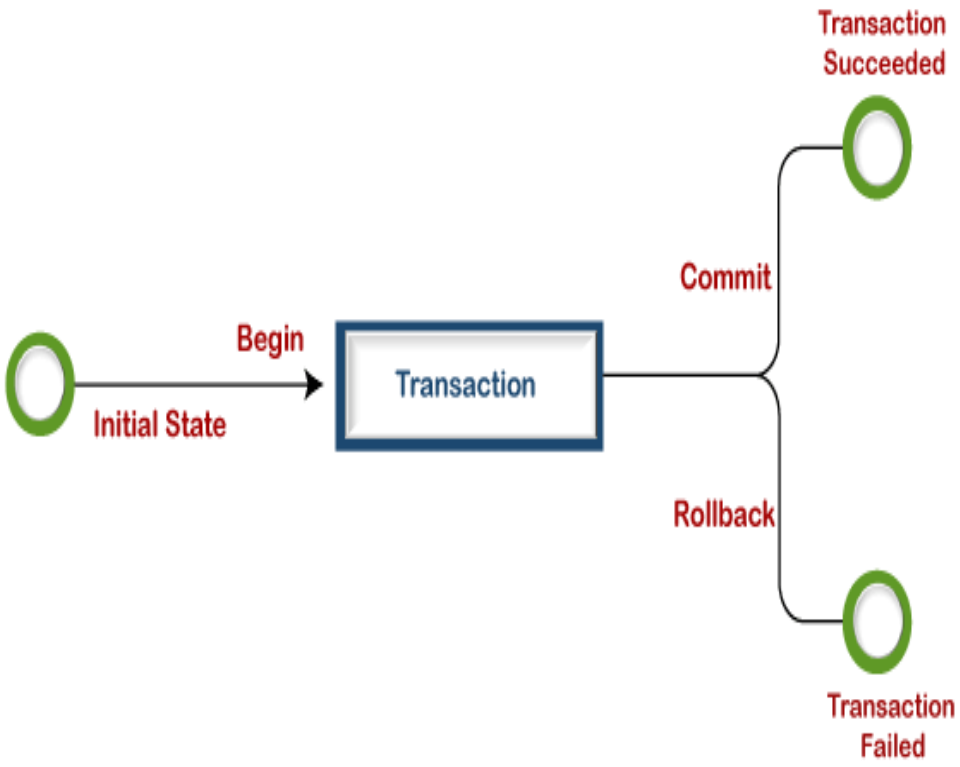
- **ROLLBACK**

  Revierte todos los cambios realizados durante la transacción. Se usa cuando ocurre un error o una condición no deseada.

Manejo de Transacciones

Universidad Tecnológica del Perú

DÉFINICIÓN



BEGIN TRANSACTION es la instrucción que inicia una transacción explícita en SQL Server. A partir de ese punto, todas las operaciones SQL ejecutadas (como INSERT, UPDATE, DELETE) quedan agrupadas como una sola unidad lógica de trabajo, que solo se hará efectiva si se ejecuta COMMIT, o se cancelará con ROLLBACK.

Sintaxis:

```
BEGIN TRANSACTION;
    -- operaciones SQL aquí
COMMIT;     -- o ROLLBACK;
```

Manejo de Transacciones

Universidad Tecnológica del Perú

BEGIN TRANSACTION

```sql
BEGIN TRANSACTION;

    UPDATE Products
    SET UnitsInStock = UnitsInStock - 5
    WHERE ProductID = 1;

    UPDATE Products
    SET UnitsInStock = UnitsInStock + 5
    WHERE ProductID = 2;

-- Si todo va bien
COMMIT;
-- Si algo sale mal, se usaría:
-- ROLLBACK;
```

FINALIDAD PRINCIPAL

- Controlar consistencia y atomicidad.
- Proteger la integridad de los datos ante errores o interrupciones.
- Permitir confirmar o revertir múltiples operaciones como una sola unidad.

Manejo de Transacciones

Universidad Tecnológica del Perú

COMMIT

```sql
BEGIN TRANSACTION;

   UPDATE Products
   SET UnitsInStock = UnitsInStock - 5
   WHERE ProductID = 1;


COMMIT; -- Todos los cambios quedan guardados
```

La sentencia **COMMIT** se utiliza para confirmar y guardar permanentemente todos los cambios realizados en la base de datos durante una transacción activa. Una vez ejecutada, los cambios no se pueden deshacer.

Se utiliza para:

- ✓ Garantizar que una serie de operaciones relacionadas se ejecuten como una sola unidad.
- ✓ Asegurar la consistencia y persistencia de los datos.
- ✓ Controlar el punto en el que los cambios se hacen visibles a otros usuarios.

Manejo de Transacciones

Universidad Tecnológica del Perú

ROLLBACK

```
UPDATE Products
    -- Supongamos que esto no es válido
    SET UnitsInStock = UnitsInStock - 1000
    WHERE ProductID = 1;

-- Decidimos cancelar
ROLLBACK;
```

La sentencia **ROLLBACK** se utiliza para revertir (deshacer) todos los cambios realizados en la base de datos desde el inicio de una transacción iniciada con BEGIN TRANSACTION. Se emplea cuando ocurre un error o cuando se desea cancelar el conjunto de operaciones antes de que sean confirmadas..

Características:

- ✓ Revierte los cambios hechos durante la transacción activa.
- ✓ Solo se puede usar dentro de una transacción iniciada con BEGIN TRANSACTION.
- ✓ Protege la integridad y consistencia de los datos ante errores inesperados.

Universidad Tecnológica del Perú

BEGIN TRY – BEGIN CATCH

Sintaxis:

```
BEGIN TRY
    -- Código que podría lanzar un error
END TRY

BEGIN CATCH
    -- Código a ejecutar si ocurre un error
END CATCH
```

Es una estructura de control de errores en SQL Server que permite capturar excepciones (errores en tiempo de ejecución) durante la ejecución de instrucciones SQL. Funciona de forma similar a los bloques try-catch en lenguajes de programación como C# o Java.

Su uso es importante porque:

- ✓ Permite detectar errores sin detener la ejecución completa del proceso.
- ✓ Facilita manejar errores personalizados (por ejemplo, con ROLLBACK, PRINT, registros de error, etc.).
- ✓ Es fundamental cuando trabajas con transacciones para evitar dejar la base de datos en un estado inconsistente.

Manejo de Transacciones

Universidad Tecnológica del Perú

EJEMPLO

```sql
BEGIN TRY
    BEGIN TRANSACTION;

    -- Reducir stock
    UPDATE Products
    SET UnitsInStock = UnitsInStock - 1
    WHERE ProductID = 1;

    -- Insertar orden
    INSERT INTO Orders (CustomerID, OrderDate)
    VALUES ('ALFKI', GETDATE());

    COMMIT;
END TRY
BEGIN CATCH
    ROLLBACK;
    PRINT 'Ocurrió un error: ' + ERROR_MESSAGE();
END CATCH;
```

```sql
BEGIN TRY
    BEGIN TRANSACTION;

    DELETE FROM Orders WHERE OrderID = 10248;
    DELETE FROM OrderDetails WHERE OrderID = 10248;

    COMMIT;
END TRY
BEGIN CATCH
    ROLLBACK;
    PRINT 'Error: ' + ERROR_MESSAGE();
END CATCH;
```

Manejo de Transacciones

Universidad Tecnológica del Perú

EJEMPLO

```sql
BEGIN TRY
    BEGIN TRANSACTION;

    UPDATE Products
    SET UnitsInStock = UnitsInStock - 10
    WHERE ProductID = 1;

    -- Error: columna no existe
    UPDATE Products
    SET StockFinal = 50
    WHERE ProductID = 1;

    COMMIT;
END TRY

BEGIN CATCH
    ROLLBACK;

    PRINT 'Error detectado: ' + ERROR_MESSAGE();
    PRINT 'Número de error: ' + CAST(ERROR_NUMBER() AS VARCHAR);
    PRINT 'Línea del error: ' + CAST(ERROR_LINE() AS VARCHAR);
END CATCH;
```

Donde se utiliza:

- En procedimientos almacenados para controlar fallos.
- Durante transacciones (BEGIN TRANSACTION) para asegurar rollback seguro.
- En scripts de mantenimiento para detectar errores sin detener toda la ejecución.

Manejo de Transacciones

Universidad Tecnológica del Perú

EJEMPLO

```sql
BEGIN TRY
    BEGIN TRANSACTION;

    UPDATE Products
    SET UnitsInStock = UnitsInStock - 10
    WHERE ProductID = 1;

    -- Error: columna no existe
    UPDATE Products
    SET StockFinal = 50
    WHERE ProductID = 1;

    COMMIT;
END TRY

BEGIN CATCH
    ROLLBACK;

    PRINT 'Error detectado: ' + ERROR_MESSAGE();
    PRINT 'Número de error: ' + CAST(ERROR_NUMBER() AS VARCHAR);
    PRINT 'Línea del error: ' + CAST(ERROR_LINE() AS VARCHAR);
END CATCH;
```

Donde se utiliza:

- En procedimientos almacenados para controlar fallos.
- Durante transacciones (BEGIN TRANSACTION) para asegurar rollback seguro.
- En scripts de mantenimiento para detectar errores sin detener toda la ejecución.

Procedimientos Almacenados

Universidad Tecnológica del Perú

Ejemplos

```
CREATE PROCEDURE AjustaPrecUnit2(
    @Ajuste decimal(10,4),
    @CodProd int
) AS
BEGIN
    DECLARE @Factor money;
    SET @Factor = 1.1;

    UPDATE Products
    SET UnitPrice = UnitPrice*@Ajuste*@Factor
    WHERE ProductID = @CodProd;

    SET @Factor = 15.0/0;

    UPDATE OrderDetails
    SET UnitPrice = UnitPrice*@Ajuste*@Factor
    WHERE ProductID = @CodProd;
END;
```

Cuando uno ejecuta un store procedure normal si hay un error a la mitad, lo que está antes del error se ejecuta pero lo que esta después no.

Procedimientos Almacenados

Universidad Tecnológica del Perú

Ejemplos

```
CREATE PROCEDURE AjustaPrecUnit3(
    @Ajuste decimal(10,4),
    @CodProd int
) AS
BEGIN
    BEGIN TRANSACTION;
    SAVE TRANSACTION puntoDeSalvado;

    DECLARE @Factor money;
    SET @Factor = 1.1;

    BEGIN TRY
        UPDATE Products
        SET UnitPrice = UnitPrice*@Ajuste*@Factor
        WHERE ProductID = @CodProd;

        SET @Factor = 15.0/0;

        UPDATE OrderDetails
        SET UnitPrice = UnitPrice*@Ajuste*@Factor
        WHERE ProductID = @CodProd;

        COMMIT TRANSACTION
    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0
        BEGIN
            ROLLBACK TRANSACTION puntoDeSalvado;
        END
    END CATCH
END;
```

- Inicio de transacción
- Punto de salvado como en los videojuegos
- Bloque (TRY) donde pueden ocurrir errores
- Se fuerza error
- Fin de transacción y graba todos los cambios
- Bloque (CATCH) donde se gestionan los errores
- Fin de transacción y deshace cambios hasta punto de salvado

Convertiremos el procedimiento almacenado anterior en uno que se maneje como una transacción, la cual ejecuta todo o nada, es decir que impide ejecuciones parciales.

Procedimientos Almacenados

Universidad Tecnológica del Perú

Ejemplos

```sql
CREATE PROCEDURE RegistrarPedido
    @CustomerID NCHAR(5),
    @OrderDate DATE,
    @Freight MONEY
AS
BEGIN
    BEGIN TRY
        BEGIN TRANSACTION;

        IF NOT EXISTS (SELECT 1 FROM Customers WHERE CustomerID = @CustomerID)
        BEGIN
            THROW 50001, 'El cliente no existe.', 1;
        END

        INSERT INTO Orders (CustomerID, OrderDate, Freight)
        VALUES (@CustomerID, @OrderDate, @Freight);

        COMMIT;
    END TRY
    BEGIN CATCH
        ROLLBACK;
        PRINT 'Error: ' + ERROR_MESSAGE();
    END CATCH
END;
```

Registrar un nuevo pedido solo si el cliente existe.

Procedimientos Almacenados

Universidad Tecnológica del Perú

Ejemplos

```sql
CREATE PROCEDURE ActualizarPrecioProducto
    @ProductID INT,
    @NuevoPrecio MONEY
AS
BEGIN
    BEGIN TRY
        BEGIN TRANSACTION;

        DECLARE @stock INT;
        SELECT @stock = UnitsInStock FROM Products WHERE ProductID = @ProductID;

        IF @stock < 1
            THROW 50002, 'El producto no tiene stock.', 1;

        UPDATE Products
        SET UnitPrice = @NuevoPrecio
        WHERE ProductID = @ProductID;

        COMMIT;
    END TRY
    BEGIN CATCH
        ROLLBACK;
        PRINT 'Error: ' + ERROR_MESSAGE();
    END CATCH
END;
```

Actualizar el precio de un producto si su stock es suficiente.

Procedimientos Almacenados

Universidad Tecnológica del Perú

Ejemplos

```sql
CREATE PROCEDURE EliminarCliente
    @CustomerID NCHAR(5)
AS
BEGIN
    BEGIN TRY
        BEGIN TRANSACTION;

        IF EXISTS (SELECT 1 FROM Orders WHERE CustomerID = @CustomerID)
        BEGIN
            THROW 50003, 'No se puede eliminar el cliente porque tiene pedidos.', 1;
        END

        DELETE FROM Customers WHERE CustomerID = @CustomerID;

        COMMIT;
    END TRY
    BEGIN CATCH
        ROLLBACK;
        PRINT 'Error: ' + ERROR_MESSAGE();
    END CATCH
END;
```

Eliminar un cliente solo si no tiene pedidos registrados.

Procedimientos Almacenados

Universidad Tecnológica del Perú

Ejemplos

```sql
CREATE PROCEDURE TransferirStock
    @OrigenID INT,
    @DestinoID INT,
    @Cantidad INT
AS
BEGIN
    BEGIN TRY
        BEGIN TRANSACTION;

        DECLARE @stockOrigen INT;
        SELECT @stockOrigen = UnitsInStock FROM Products WHERE ProductID = @OrigenID;

        IF @stockOrigen < @Cantidad
            THROW 50004, 'Stock insuficiente en producto origen.', 1;

        UPDATE Products
        SET UnitsInStock = UnitsInStock - @Cantidad
        WHERE ProductID = @OrigenID;

        UPDATE Products
        SET UnitsInStock = UnitsInStock + @Cantidad
        WHERE ProductID = @DestinoID;

        COMMIT;
    END TRY
    BEGIN CATCH
        ROLLBACK;
        PRINT 'Error en transferencia: ' + ERROR_MESSAGE();
    END CATCH
END;
```

Transferir stock entre dos productos.

Procedimientos Almacenados

Universidad Tecnológica del Perú

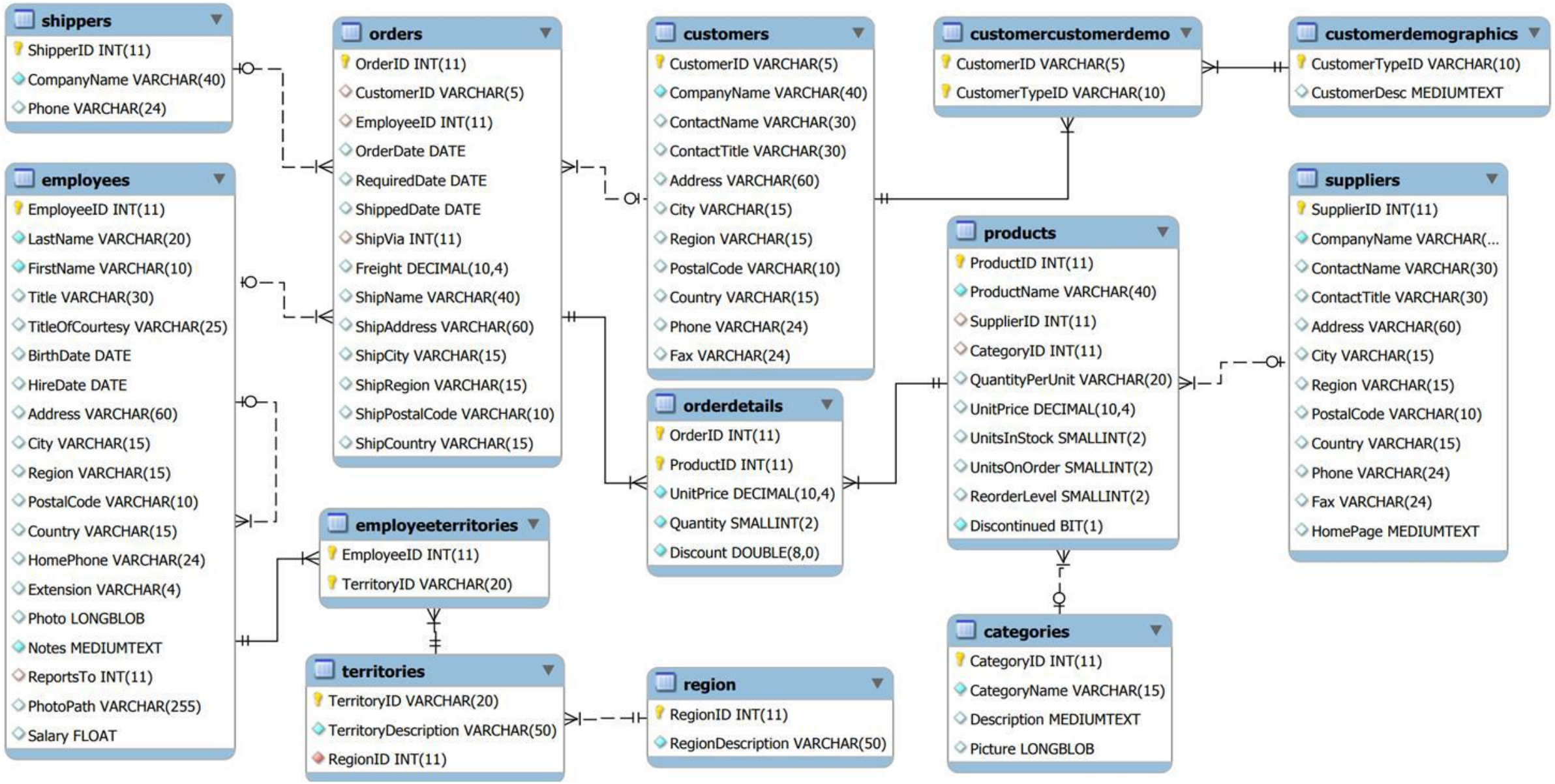
Ejemplos

```sql
CREATE PROCEDURE AsignarEmpleadoPedido
    @OrderID INT,
    @EmployeeID INT
AS
BEGIN
    BEGIN TRY
        BEGIN TRANSACTION;

        DECLARE @asignado INT;
        SELECT @asignado = EmployeeID FROM Orders WHERE OrderID = @OrderID;

        IF @asignado IS NOT NULL
            THROW 50005, 'El pedido ya tiene un empleado asignado.', 1;

        UPDATE Orders
        SET EmployeeID = @EmployeeID
        WHERE OrderID = @OrderID;

        COMMIT;
    END TRY
    BEGIN CATCH
        ROLLBACK;
        PRINT 'Error: ' + ERROR_MESSAGE();
    END CATCH
END;
```

Asignar empleado a pedido solo si el pedido aún no está asignado.

EJERCICIOS

Universidad Tecnológica del Perú



Cierre

Diferencia entre commit y rollback.

Cuál es la finalidad de try catch

Universidad Tecnológica del Perú